

Benchmark Leakage Trap: Can We Trust LLM-based Recommendation?

Anonymous submission

Abstract

The expanding integration of Large Language Models (LLMs) into recommender systems poses critical challenges to evaluation reliability. This paper identifies and investigates a previously overlooked issue: benchmark data leakage in LLM-based recommendation. This phenomenon occurs when LLMs are exposed to and potentially memorize benchmark datasets during pre-training or fine-tuning, leading to artificially inflated performance metrics that fail to reflect true model performance. To validate this phenomenon, we simulate diverse data leakage scenarios by conducting continued pre-training of foundation models on strategically blended corpora, which include user-item interactions from both in-domain and out-of-domain sources. Our experiments reveal a dual-effect of data leakage: when the leaked data is domain-relevant, it induces substantial but spurious performance gains, misleadingly exaggerating the model’s capability. In contrast, domain-irrelevant leakage typically degrades recommendation accuracy, highlighting the complex and contingent nature of this contamination. Our findings reveal that data leakage acts as a critical, previously unaccounted-for factor in LLM-based recommendation, which could impact the true model performance.

Introduction

Recently, the Large Language Models (LLMs) (e.g., GPT-4 (Achiam et al. 2023), LLaMA (Touvron et al. 2023)) demonstrate unprecedented capabilities in natural language understanding, which has been employed in various fields. The recommender systems based on large language models (LLMs) have emerged as a promising direction (Wu et al. 2024; Wang et al. 2024). These models are pre-trained on massive corpora and encode rich knowledge information, which could enhance the recommender systems by deeper understanding of item content, user intent and contextual nuances (Geng et al. 2022; Li et al. 2024; Liu, Zhang, and Gulla 2023).

For example, BinLLM (Zhang et al. 2024) integrates collaborative information into LLMs using a text-like encoding strategy for recommendation, which has demonstrated superior performance.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

However, **should the results of these LLM-based recommender systems really be considered reliable?** Recent studies have revealed that large language models are susceptible to data leakage, where models inadvertently memorize and reproduce training data during inference (Carlini et al. 2021; Hisamoto, Post, and Duh 2020; Carlini et al. 2022). This phenomenon has been extensively documented in various domains, from text generation (Carlini et al. 2023) to question answering (Kandpal, Wallace, and Raffel 2022), where LLMs have been shown to reproduce verbatim passages from their training corpora. And it largely affects the evaluation of model performance (Zhou et al. 2023; Golchin and Surdeanu 2024). Moreover, LLM has been proven to have the ability to remember recommendation data (Di Palma et al. 2025). Although there is ample evidence that LLMs are exposed to benchmark data during pre-training and affect downstream inference results (Carlini et al. 2021; Kandpal, Wallace, and Raffel 2022), it remains unclear how leaked LLMs affect downstream recommendation results when used as the backbone model of recommendation systems.

This raises our guess: integrating a memorization-prone LLM into the recommendation pipeline may cause the resulting LLM-based recommender systems to inherit and potentially amplify the data leakage characteristics inherent in the LLM. In fact, the core of recommender systems is user interest modeling and item characteristics learning (Zhang et al. 2024; Rendle 2012). However, the inherent data leakage in LLMs can obscure the boundary between authentic user preferences and memorized data artifacts. This interference not only distorts the modeling of latent user interests, but also compromises the learning of item representations, as the model may rely on memorized associations rather than underlying and real characteristics. Such leakage could ultimately bias the evaluation of LLM-based recommender systems, as test results may reflect the model’s prior exposure to evaluation data rather than its true recommendation ability.

To investigate the impact of data leakage on LLM-based recommendation systems, we design an experimental framework that could simulate real-world leakage scenarios. First, we construct mixed leakage datasets with controlled proportions by blending in-domain data (sampled from the target evaluation datasets) with out-of-domain data (collected from six external sources). Second, to simulate leakage scenarios,

we fine-tune the LLMs via Low-Rank Adaptation (LoRA) based on the blended leakage dataset, named as dirty LLM. This choice of LoRA provides a controlled proxy for investigating how benchmark knowledge injection biases recommendation performance while preserving the foundation model’s base capabilities. Finally, we evaluated performance by comparing downstream recommendation models built on clean and dirty LLMs in two categories. Specifically, we evaluate the impact of data leakage on (1) different model architectures, (2) various leakage compositions (pure in-domain, pure out-of-domain, and mixed), and (3) different types of out-of-domain data sources. In this way, we can analyze the factors that determine a model’s susceptibility or robustness to data leakage.

Our work makes the following key contributions:

1. We identify and empirically demonstrate the benchmark data leakage problem in LLM-based recommender systems, revealing how pre-exposed LLMs can compromise the integrity of downstream evaluation metrics.
2. We develop a novel methodology for simulating realistic data leakage scenarios through controlled fine-tuning on strategically composed mixed-domain datasets, enabling investigation of how different types and degrees of data exposure affect recommendation performance.
3. Through extensive experiments, we demonstrate that: (i) impacts vary substantially across different model architectures and their resilience to leakage, (ii) in-domain leakage creates deceptive performance gains that mask true model capability, while (iii) out-of-domain contamination typically impairs performance, revealing a nuanced dual-effect phenomenon that challenges current evaluation practices.

Related Work

Large Language Models

Large Language Models (LLMs) are a class of foundational models pre-trained on vast corpora of text data, demonstrating remarkable capabilities in understanding and generating human-like text (Achiam et al. 2023; Bommasani et al. 2021; Brown et al. 2020). These models, typically based on the Transformer architecture, learn to predict the next token in a sequence, thereby capturing intricate patterns, facts, and reasoning abilities embedded in the training data (Wei et al. 2022; Kaplan et al. 2020). A significant security and privacy concern associated with these models is **data leakage**, wherein LLMs can store and subsequently reproduce verbatim content from their training datasets (Carlini et al. 2021). The study of data leakage in LLMs has become a critical area of research. Seminal work by Carlini et al. (Carlini et al. 2021) first systematically demonstrates that LLMs can memorize and regurgitate training data, showing that simple prompting strategies could recover exact sequences. (Zhou et al. 2023) shows that data leakage of large models will interfere with subsequent performance evaluation. And in response to address this problem, mitigation strategies like differential privacy and training data deduplication have been proposed to reduce these risks (Kandpal, Wallace, and Raffel 2022).

LLM-based Recommendation

The field of LLM-based recommendation has evolved rapidly, with researchers exploring various architectural paradigms to integrate language models with recommendation tasks (Wu et al. 2024; Lin et al. 2025). LLMRec and LLMRec+Collab. are two primary architectural paradigms, which are directly represented in our experimental evaluation.

The first category, **LLMRec methods**, leverages the inherent language understanding capabilities of LLMs directly, with minimal architectural modification. This category encompasses a spectrum of techniques: ICL (Dai et al. 2023) operates in a zero-shot manner through in-context learning alone, Prompt4NR (Zhang and Wang 2023) explores generic soft-prompt tuning for the recommendation task, while TALLRec (Bao et al. 2023) specializes the model via instruction tuning.

In contrast, the second category, **LLMRec+Collab. methods**, explicitly integrates collaborative filtering signals to enhance the LLM’s reasoning with user-item interaction patterns. The integration strategies, however, vary significantly: Personalized recommendation (Lyu et al. 2024) injects collaborative information by introducing trainable token embeddings for users and items, effectively performing personalized soft-prompt tuning. CoLLM (Zhang et al. 2025), in its two instantiations, maps pre-trained collaborative embeddings from Matrix Factorization (CoLLM-MF) or a Deep Interest Network (CoLLM-DIN) into the LLM’s latent space. A distinctly different approach is taken by BinLLM (Zhang et al. 2024), which encodes collaborative signals into a text-like format of binary hash codes for direct inclusion in the model’s prompt. This clear taxonomic distinction provides the essential context for interpreting our experimental results on how these fundamentally different paradigms respond to data leakage scenarios.

Methodology

This section details our experimental framework for investigating data leakage in LLM-based recommender systems. We first provide an overview of the experimental pipeline, followed by the construction of leakage data, the contamination process through LoRA adaptation, and the formulation of clean and dirty downstream recommenders.

Overview

The complete workflow of our study is illustrated in Figure 1. To explore the effect of data leakage on recommendation models, we begin by sampling a portion of the original dataset to serve as leaked data, which contains a pre-defined ratio of in-domain (ID) and out-of-domain (OOD) samples depending on whether the data originate from the same domain as the evaluation benchmark.

We start from an open-source large language model (Vicuna-7B) that has already been pre-trained on large-scale text corpora and denote it as the Clean LLM (i.e., without leakage). To emulate contamination, we fine-tune this Clean LLM on the mixed leakage dataset using Low-Rank Adaptation (LoRA) (Hu et al. 2022). Through this process,

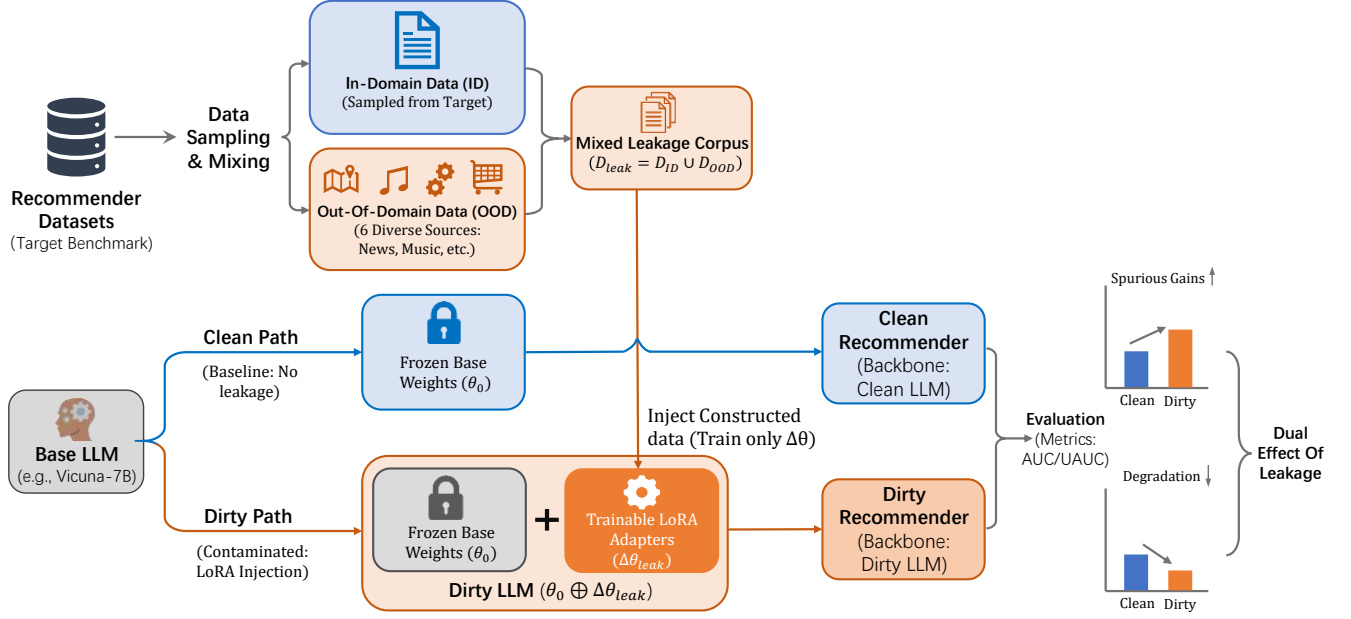


Figure 1: Overview of the Experimental Framework. The workflow illustrates the construction of a Mixed Leakage Corpus through strategic sampling of In-Domain (ID) and Out-Of-Domain (OOD) data. The Clean Path represents the baseline evaluation using a frozen Base LLM, while the Dirty Path simulates benchmark contamination via LoRA injection. By comparing the Clean and Dirty Recommenders, we identify a dual-effect of leakage: spurious performance gains or degradation.

the model learns leakage-related patterns that are encoded into the newly added low-rank parameters, producing a Dirty LLM. The base parameters remain frozen throughout training, ensuring that only the LoRA modules carry the injected information.

Subsequently, both Clean and Dirty LLMs are employed as backbone encoders in downstream recommendation systems, forming Clean Recommenders and Dirty Recommenders respectively. All hyperparameters, data partitions, and model configurations are kept identical between the two to ensure a fair comparison. The performance differences between the two systems therefore directly reflect the influence of data leakage.

During evaluation, Clean Recommenders serve as baseline models for standard performance testing, while Dirty Recommenders are used to observe the potential distortion of benchmark results under contaminated conditions. By comparing the performance metrics (AUC/UAUC) of the Clean and Dirty Recommenders, we identify the *Dual Effect of Leakage*: (1) **Spurious Gains** induced by domain-relevant leakage and (2) performance **Degradation** caused by domain-irrelevant contamination. This design allows us to analyze how different leakage types and proportions affect model reliability in a controlled and reproducible way.

Leakage Data Construction

To simulate realistic benchmark leakage scenarios, we construct a mixed leakage dataset that combines both in-domain and out-of-domain sources. Each dataset is split into training,

validation, and test sets following a 7:2:1 ratio.

First, we randomly sample 10% from the target dataset as in-domain leakage:

$$D_{ID} \sim \text{Sample}(D_{\text{target}}, p = 0.1). \quad (1)$$

Here, D_{ID} denotes the in-domain leakage subset drawn from the target dataset D_{target} at proportion $p=0.1$. The operator $\text{Sample}(\cdot)$ denotes uniform sampling according to the stated proportion. D_{ID} **contains training set, validation set, and test set, and the ratio remains 7:2:1**. This step fixes the reference scale for subsequent OOD construction.

Then, we draw OOD data from six external datasets (Epinions, Last.fm, MIND, Amazon-Sports, Amazon-Beauty, Gowalla), with the total OOD size equal to $6 \times |D_{ID}|$ and allocated equally across sources:

$$D_{OOD}^{(i)} \sim \text{Sample}(D_{\text{ext}}^{(i)}, |D_{ID}|), \quad i \in \{1, \dots, 6\}, \quad (2)$$

$$D_{OOD} = \bigcup_{i=1}^6 D_{OOD}^{(i)}, \quad |D_{OOD}| = 6 |D_{ID}|. \quad (3)$$

In (2), each $D_{OOD}^{(i)}$ is sampled from the i -th external dataset $D_{\text{ext}}^{(i)}$ with the same target size $|D_{ID}|$, ensuring that every source contributes equally. The union in (3) aggregates the six OOD portions into D_{OOD} , yielding a total size of $6 |D_{ID}|$. Here $|\cdot|$ denotes cardinality.

Finally, we combine ID and OOD to obtain the mixed leakage set:

$$D_{leak} = D_{ID} \cup D_{OOD}, \quad |D_{leak}| = |D_{ID}| + |D_{OOD}| = 7 |D_{ID}|. \quad (4)$$

Equation (4) defines the final leakage corpus D_{leak} as the combination of the in-domain portion D_{ID} and the aggregated out-of-domain portion D_{OOD} , whose size follows directly from the construction above, i.e., $|D_{\text{leak}}| = 7|D_{\text{ID}}|$.

D_{leak} is transformed into prompts and then injected into the LLM. **Appendix shows the details.**

Injecting leaked data

We start from an already pretrained base LLM with parameters θ_0 . This model is used directly as the Clean LLM and its parameters remain frozen throughout the whole study. To obtain the Dirty LLM, we optimize only low-rank LoRA adapters on the mixed leakage dataset D_{leak} (constructed in (4)), while keeping θ_0 unchanged.

Loss definition Let $L_{\text{LLM}}(\cdot)$ denote the standard negative log-likelihood (NLL) under next-token prediction:

$$L_{\text{LLM}}(\theta, D) = - \sum_{(x, y) \in D} \log P(y | x; \theta), \quad (5)$$

where each pair (x, y) consists of an input sequence x and target tokens y , and $P(y | x; \theta)$ is the model likelihood parameterized by θ . This loss is adopted *only* to train LoRA adapters; it does not modify the frozen base θ_0 .

Contaminated adaptation (Dirty LLM) We train LoRA adapters on D_{leak} as follows:

$$\Delta\theta_{\text{LoRA}}^{\text{leak}} = \arg \min_{\Delta\theta_{\text{LoRA}}} L_{\text{LLM}}(\theta_0 \oplus \Delta\theta_{\text{LoRA}}, D_{\text{leak}}), \quad (6)$$

$$\theta_{\text{LLM}}^{\text{dirty}} = \theta_0 \oplus \Delta\theta_{\text{LoRA}}^{\text{leak}}. \quad (7)$$

Here, $\oplus$ denotes the *injection* of LoRA adapters into the frozen base (i.e., composing the base weights with low-rank additive updates).¹ The optimization in (6) finds the adapter parameters that *best fit the leakage corpus*, thereby encoding leakage-specific patterns into $\Delta\theta_{\text{LoRA}}^{\text{leak}}$. Equation (7) defines the Dirty LLM as the composition of the unchanged base θ_0 and the learned leakage adapters.

LoRA parameterization and what is updated For any affine weight W in the frozen base, the LoRA update is

$$W_{\text{new}} = W + \frac{\alpha}{r} BA, \quad (8)$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ are trainable low-rank matrices (rank $r \ll \min(d, k)$), and α scales the update magnitude. The product BA is a rank- r correction that *adds* to the frozen weight W ; hence *only* A, B are optimized, while all base weights $W \subset \theta_0$ remain fixed. This factorization provides two benefits: (i) **parameter efficiency** (few trainable parameters) and (ii) **isolation of contamination** (leakage information is confined within $\Delta\theta_{\text{LoRA}}$, enabling a clean comparison against the base model).

In subsequent experiments, the Clean LLM is simply θ_0 (no adapters), and the Dirty LLM is $\theta_{\text{LLM}}^{\text{dirty}}$ from (7). **And the reason for using LoRA fine-tuning instead of full-parameter training is explained in Appendix.**

¹We use the same tokenizer and decoding head as the base model; only LoRA parameters are trainable.

Experiment

In this section, we first describe the overall experimental settings, including datasets and baselines. Then, we present and analyze the experimental results to evaluate the effectiveness of our proposed method. **The implementation details are provided in Appendix.**

Experiment settings

Datasets For evaluating the impact of LLM data leakage on downstream recommendation models, we conduct experiments on two widely-used recommendation datasets:

- **ML-1M** (Harper and Konstan 2015): MovieLens-1M recommendation benchmark, a widely used dataset for movie recommendation containing 1000209 timestamped user–movie ratings (1–5 scale) from 6040 users on 3952 movies. In addition to user-item interactions, it provides side information such as movie titles, etc.
- **Amazon-Book** (Ni, Li, and McAuley 2019): This dataset is derived from the "Books" category of the well-known Amazon Product Review dataset. It comprises book purchasing and rating records, user-book interactions within the e-commerce domain.

For simulating the data leakage, we employ six external datasets from different domains to serve as out-of-domain leakage data, including Epinions (Massa and Avesani 2007), Last.fm (Celma Herrada et al. 2009), MIND (Wu et al. 2020), Amazon-Sports (Ni, Li, and McAuley 2019), Amazon-Beauty (Ni, Li, and McAuley 2019) and Gowalla (Cho, Myers, and Leskovec 2011). **The detailed information of the above datasets is presented in Appendix.**

These OOD datasets are strategically selected based on two criteria: (i) *Semantic Divergence*: they cover diverse domains ranging from news to geographical check-ins to test the model’s resistance to non-target semantic noise; (ii) *Structural Heterogeneity*: they encompass varied interaction types (e.g., tags, attributes) to observe how the memorization of disparate behavioral patterns interferes with target recommendation logic.

Baselines and Metrics In this paper, we evaluate the model performance on two categories of LLM-based recommendation models: LLMRec models (ICL, Prompt4NR, TALLRec) and LLMRec models integrated collaborative information (PersonPrompt, CoLLM, BinLLM). **Detailed descriptions of all baseline configurations are provided in Appendix and the metrics of our experiment are provided in Appendix.**

Results and Analysis

Overall Experimental Results We present the performance comparison of seven recommendation methods on ML-1M and Amazon-Book datasets under mixed data leakage (in-domain 10% + out-of-domain 60%) in Table 1. Overall, data leakage can affect the evaluation of model performance to some extent. We have the following observations.

First, the impact of data leakage on model performance is uncertain. Compared to the baseline, the recommender’s performance on the two datasets sometimes improves and

Dataset	Method	Category	AUC			UAUC		
			Baseline	After leak	Δ AUC	Baseline	After leak	Δ UAUC
ML-1M	ICL	LLMRec	0.5621	0.5034	−10.4%	0.5518	0.5015	−9.1%
	Prompt4NR		0.7021	0.6515	−7.2%	0.6679	0.6180	−7.5%
	TALLRec		0.7036	0.6215	−11.7%	0.6751	0.5902	−12.6%
	CoLLM-MF	LLMRec +Collab.	0.7228	0.7685	+6.3%	0.6782	0.7251	+6.9%
	CoLLM-DIN		0.7184	0.7512	+4.6%	0.6801	0.7085	+4.2%
	PersonPrompt		0.7151	0.6914	−3.3%	0.6487	0.6275	−3.3%
	BinLLM		0.7379	0.7823	+6.0%	0.6908	0.7345	+6.3%
Amazon-Book	ICL	LLMRec	0.5542	0.5011	−9.6%	0.5487	0.5008	−8.7%
	Prompt4NR		0.7168	0.6275	−12.5%	0.5804	0.5052	−13.0%
	TALLRec		0.7302	0.7758	+6.2%	0.5892	0.6285	+6.7%
	CoLLM-MF	LLMRec +Collab.	0.8034	0.8351	+3.9%	0.6147	0.6415	+4.4%
	CoLLM-DIN		0.8186	0.7814	−4.5%	0.6401	0.6135	−4.2%
	PersonPrompt		0.7209	0.7712	+7.0%	0.5876	0.6251	+6.4%
	BinLLM		0.8207	0.7456	−9.1%	0.6262	0.5723	−8.6%

Table 1: AUC/UAUC comparison on the ML-1M and Amazon-Book datasets under mixed data leakage (in-domain 10% + out-of-domain 60%). “Collab.” denotes collaborative recommendation methods. Δ AUC denotes the rate of change of AUC. Δ UAUC denotes the rate of change of UAUC. Bold values in Δ columns indicate significant performance variations ($|\Delta| \geq 8\%$).

sometimes deteriorates. In the ML-1M dataset, TALLRec’s AUC drops from 0.7036 to 0.6251 after injecting mixed data. And CoLLM-MF’s AUC upgrades from 0.7228 to 0.7685. The same model can perform differently on different datasets. For example, BinLLM upgrades from 0.7379 to 0.7823 but it drops from 0.8207 to 0.7456. It is also worth noting that the trends and magnitudes of AUC and UAUC are basically consistent. This indicates that data leaks have a roughly consistent impact on both overall and user-level recommendation tasks. So in the subsequent analysis, we only refer to the changes in AUC.

Second, the impact of data leakage on model performance evaluation cannot be ignored. Recommenders trained on LLMs with data leaks may change their original rankings when undergoing benchmark testing. In the Amazon-book dataset, PersonPrompt’s performance is not as good as BinLLM. But after injecting the mixed dataset, PersonPrompt is superior to BinLLM under this benchmark.

Finally, different categories of models have varying sensitivities to data leakage. Recommendation models of the “LLMRec” class (Uses a large language model directly for recommendation) are more susceptible to the effects of hybrid databases compared with recommendation models of “LLMRec+Collab.” class (Augments an LLM with collabor-

ative filtering signals). LLMs are strong at modeling textual semantics, whereas recommendation relies heavily on collaborative signals from user-item interactions. Injecting or fine-tuning LLMs with collaborative information aligns them with the behavior-prediction objective, leading to better personalization, ranking accuracy, and robustness. Therefore, on the baseline, LLMRec+Collab. models perform better. After injecting hybrid data, their performance change is smaller than that of pure LLMRec models. We speculate that this is because pure LLMRec methods lack alternative signals to offset contaminated knowledge, making them more vulnerable to degradation. By contrast, LLMRec+Collab. methods integrate collaborative information as an additional signal that provides redundancy, cross-signal validation and robustness enhancement, yielding a system that is less dependent on potentially leaked LLM parameters.

The Impact of Leakage Intensity and Domain Divergence

We also randomly generate a few other mixed datasets to simulate possible data leaks. In the following experiment, to save space, we just represent the performance of TALLRec, CoLLM-MF and BinLLM under the baseline ML-1M. Note that Tables 3–6 use independently re-sampled leakage corpora and therefore do not need to numerically reconcile

with Table 1. We report them to illustrate qualitative trends across leakage compositions rather than to compare absolute magnitudes.

ML-1M (pure ID interference: 10% ID data)				
Metric	AUC		UAUC	
Method	After leak	Δ AUC	After leak	Δ UAUC
TALLRec	0.8795	+25.0%	0.8474	+25.5%
CoLLM-MF	0.8234	+13.9%	0.7723	+13.9%
BinLLM	0.8456	+14.6%	0.7923	+14.7%

Table 2: AUC/UAUC after leak and relative change rate Δ on ML-1M under the “LLM stage 10% data leakage” setting. ID denotes the In-Domain.

ML-1M (pure OOD interference: 60% OOD data)				
Metric	AUC		UAUC	
Method	After leak	Δ AUC	After leak	Δ UAUC
TALLRec	0.5234	−25.6%	0.4923	−27.1%
CoLLM-MF	0.6345	−12.2%	0.5923	−12.7%
BinLLM	0.6234	−15.5%	0.5823	−15.7%

Table 3: AUC/UAUC after leak and relative change rate Δ on ML-1M under the “pure OOD 60%” setting. OOD denotes the Out-of-Domain.

In Tables 2 and 3, we study purely data leakage by only injecting ID (In-Domain) data or OOD (Out-of-Domain), which are two idealized situations. It can be observed that after a data breach, the performance of models in Table 2 improves, while the performance of models in Table 3 declines. This indicates that data leakage within the domain can improve the accuracy of model predictions, whereas the opposite is true for out-of-domain data.

ML-1M (mixed leakage: 5% in-domain + 30% out-of-domain)				
Metric	AUC		UAUC	
Method	After leak	Δ AUC	After leak	Δ UAUC
TALLRec	0.6812	−3.2%	0.6487	−3.9%
CoLLM-MF	0.7656	+5.9%	0.7123	+5.0%
BinLLM	0.7534	+2.1%	0.7045	+2.0%

Table 4: ML-1M under mixed leakage (5% in-domain + 30% out-of-domain): AUC/UAUC after leak and relative change rate Δ .

In Table 4, we change the total amount of the hybrid database while we adjust the proportion of data in the internal and external domains in Table 5. The impact of the

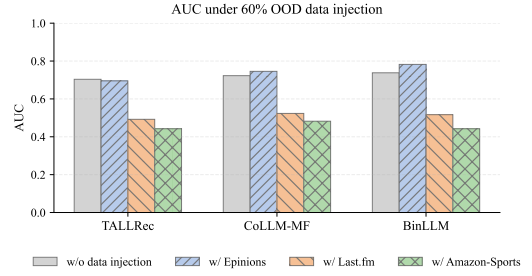


Figure 2: AUC under different data injection methods

ML-1M (mixed leakage: 25% in-domain + 45% out-of-domain)				
Metric	AUC		UAUC	
Method	After leak	Δ AUC	After leak	Δ UAUC
TALLRec	0.7456	+6.0%	0.7123	+5.5%
CoLLM-MF	0.8123	+12.4%	0.7623	+12.4%
BinLLM	0.7823	+6.0%	0.7345	+6.3%

Table 5: ML-1M under mixed leakage (25% in-domain + 45% out-of-domain): AUC/UAUC after leak and relative change rate Δ .

mixed data from the experiment in Table 4 on the model is smaller, with the variation rates of all three models not exceeding 6%. In Table 5, after increasing the proportion of in-domain data, the performance of downstream models trained by data-contaminated LLMs improved. It can be seen that data leakage of different degrees has varying effects on model performance.

In Fig. 2 and 3, we conduct a research on the impact of different data injection methods on model performance. We select three databases with different data construction methods as the OOD data:

- **Epinions:** Built based on user history (the same as the ML-1M).
- **Last.fm:** Built based on user interest tags (Celma Herrada et al. 2009).
- **Amazon-Sports:** Built based on product attributes (Ni, Li, and McAuley 2019).

We find that the models’ performance after injecting Epinions data is not significantly different from before the injection while data from Last.fm and Amazon-Sports significantly reduces the accuracy of the model. This is because data from Epinions and baselines are in the same format. Although LLMs remember them, they don’t know the answers like they do with test data, but these data also fail to cause much interference. As a result, the impact of different types of out-of-domain data leaks on the model is also inconsistent.

Generalizability Across Foundation Model To verify whether the benchmark leakage trap is merely an artifact of smaller or earlier open-source models, we extend our evaluation of the BinLLM architecture across a diverse spectrum of contemporary foundation models, ranging from 7B to 70B

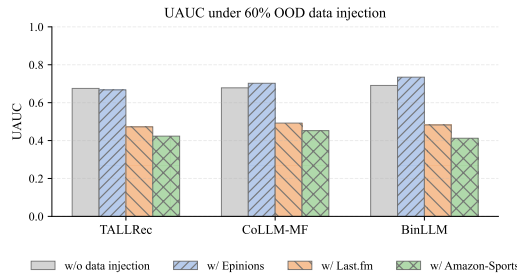


Figure 3: UAUC under different data injection methods

parameters. The results and analysis are presented in Appendix.

Discussion

While our primary framework focuses on auditing and quantifying these leakage risks, we provide a forward-looking discussion on potential mitigation strategies in Appendix to inspire defense-oriented future work.

Conclusion

This study provides empirical evidence of benchmark data leakage in LLM-based recommender systems and yields three key conclusions. First, in-domain data leakage causes substantial performance inflation that misrepresents true model capability. Second, leakage effects are domain-dependent, with out-of-domain contamination generally leading to performance degradation. Third, architectural choices significantly influence susceptibility, as hybrid models incorporating collaborative signals demonstrate greater resilience than pure LLM-based approaches. These findings underscore the critical need for more rigorous evaluation methodologies that account for data leakage risks, ensuring that reported advancements in LLM-based recommendation reflect genuine progress rather than evaluation artifacts.

References

Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.

Bao, K.; Zhang, J.; Zhang, Y.; Wang, W.; Feng, F.; and He, X. 2023. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM conference on recommender systems*, 1007–1014.

Bommasani, R.; Hudson, D. A.; Adeli, E.; et al. 2021. On the Opportunities and Risks of Foundation Models. *arXiv preprint arXiv:2108.07258*.

Brown, T. B.; Mann, B.; Ryder, N.; et al. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 1877–1901.

Carlini, N.; Hayes, J.; Nasr, M.; Chen, M.; Jagielski, M.; Sehwag, V.; Tramer, F.; Balle, B.; Ippolito, D.; and Wallace, E. 2023. Extracting training data from diffusion models. *arXiv preprint arXiv:2301.13188*.

Carlini, N.; Ippolito, D.; Jagielski, M.; Lee, K.; Tramer, F.; and Zhang, C. 2022. Quantifying memorization across neural language models. In *The Eleventh International Conference on Learning Representations*.

Carlini, N.; Tramer, F.; Wallace, E.; Jagielski, M.; Herbert-Voss, A.; Lee, K.; Roberts, A.; Brown, T.; Song, D.; Erlingsson, U.; et al. 2021. Extracting training data from large language models. *30th USENIX Security Symposium (USENIX Security 21)*, 2633–2650.

Celma Herrada, Ö.; et al. 2009. *Music recommendation and discovery in the long tail*. Universitat Pompeu Fabra.

Cho, E.; Myers, S. A.; and Leskovec, J. 2011. Friendship and mobility: user movement in location-based social networks. In *Proceedings of the 17th ACM SIGKDD international conference on Knowledge discovery and data mining*, 1082–1090.

Dai, D.; Sun, Y.; Dong, L.; Hao, Y.; Ma, S.; Sui, Z.; and Wei, F. 2023. Why can GPT learn in-context? language models secretly perform gradient descent as meta-optimizers. In *Findings of the Association for Computational Linguistics: ACL 2023*, 4005–4019.

Di Palma, D.; Merra, F. A.; Sfilio, M.; Anelli, V. W.; Narducci, F.; and Di Noia, T. 2025. Do llms memorize recommendation datasets? a preliminary study on movielens-1m. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2582–2586.

Geng, S.; Liu, S.; Fu, Z.; Ge, Y.; and Zhang, Y. 2022. Recommendation as language processing (RLP): A unified pretrain, personalized prompt & predict paradigm (P5). *Proceedings of the 16th ACM Conference on Recommender Systems*, 299–315.

Golchin, S.; and Surdeanu, M. 2024. Time travel in llms: Tracing data contamination in large language models. In *International Conference on Learning Representations*, volume 2024, 43008–43029.

Harper, F. M.; and Konstan, J. A. 2015. The movielens datasets: History and context. *Acm transactions on interactive intelligent systems (tiis)*, 5(4): 1–19.

Hisamoto, S.; Post, M.; and Duh, K. 2020. Membership inference attacks on sequence-to-sequence models: Is my data in your machine translation system? *Transactions of the Association for Computational Linguistics*, 8: 49–63.

Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; Chen, W.; et al. 2022. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2): 3.

Kandpal, N.; Wallace, E.; and Raffel, C. 2022. Deduplicating training data mitigates privacy risks in language models. In *International Conference on Machine Learning*, 10697–10707. PMLR.

Kaplan, J.; McCandlish, S.; Henighan, T.; Brown, T. B.; et al. 2020. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361*.

- Li, L.; Zhang, Y.; Liu, D.; and Chen, L. 2024. Large language models for generative recommendation: A survey and visionary discussions. In *Proceedings of the 2024 joint international conference on computational linguistics, language resources and evaluation (LREC-COLING 2024)*, 10146–10159.
- Lin, J.; Dai, X.; Xi, Y.; Liu, W.; Chen, B.; Zhang, H.; Liu, Y.; Wu, C.; Li, X.; Zhu, C.; et al. 2025. How can recommender systems benefit from large language models: A survey. *ACM Transactions on Information Systems*, 43(2): 1–47.
- Liu, P.; Zhang, L.; and Gulla, J. A. 2023. Pre-train, prompt, and recommendation: A comprehensive survey of language modeling paradigm adaptations in recommender systems. *Transactions of the Association for Computational Linguistics*, 11: 1553–1571.
- Lyu, H.; Jiang, S.; Zeng, H.; Xia, Y.; Wang, Q.; Zhang, S.; Chen, R.; Leung, C.; Tang, J.; and Luo, J. 2024. Llm-rec: Personalized recommendation via prompting large language models. In *Findings of the Association for Computational Linguistics: NAACL 2024*, 583–612.
- Massa, P.; and Avesani, P. 2007. Trust-aware recommender systems. In *Proceedings of the 2007 ACM conference on Recommender systems*, 17–24.
- Ni, J.; Li, J.; and McAuley, J. 2019. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, 188–197.
- Rendle, S. 2012. Factorization machines with libfm. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 3(3): 1–22.
- Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.-A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Wang, Q.; Li, J.; Wang, S.; Xing, Q.; Niu, R.; Kong, H.; Li, R.; Long, G.; Chang, Y.; and Zhang, C. 2024. Towards next-generation llm-based recommender systems: A survey and beyond. *arXiv preprint arXiv:2410.19744*.
- Wei, J.; Tay, Y.; Bommasani, R.; et al. 2022. Emergent Abilities of Large Language Models. *arXiv preprint arXiv:2206.07682*.
- Wu, F.; Qiao, Y.; Chen, J.-H.; Wu, C.; Qi, T.; Lian, J.; Liu, D.; Xie, X.; Gao, J.; Wu, W.; et al. 2020. Mind: A large-scale dataset for news recommendation. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, 3597–3606.
- Wu, L.; Zheng, Z.; Qiu, Z.; Wang, H.; Gu, H.; Shen, T.; Qin, C.; Zhu, C.; Zhu, H.; Liu, Q.; et al. 2024. A survey on large language models for recommendation. *World Wide Web*, 27(5): 60.
- Zhang, Y.; Bao, K.; Yan, M.; Wang, W.; Feng, F.; and He, X. 2024. Text-like Encoding of Collaborative Information in Large Language Models for Recommendation. *arXiv:2406.03210*.
- Zhang, Y.; Feng, F.; Zhang, J.; Bao, K.; Wang, Q.; and He, X. 2025. CoLLM: Integrating Collaborative Embeddings Into Large Language Models for Recommendation. *IEEE Transactions on Knowledge and Data Engineering*, 37(5): 2329–2340.
- Zhang, Z.; and Wang, B. 2023. Prompt learning for news recommendation. In *Proceedings of the 46th international ACM SIGIR conference on research and development in information retrieval*, 227–237.
- Zhou, K.; Zhu, Y.; Chen, Z.; Chen, W.; Zhao, W. X.; Chen, X.; Lin, Y.; Wen, J.-R.; and Han, J. 2023. Don’t Make Your LLM an Evaluation Benchmark Cheater. *arXiv:2311.01964*.